Lab 6 Report: Path Planning with Real Time Monte Carlo Localization in Mapped Environments

Team 13 - TAMJAM

Tissany Chen
Jeryl Lewis
Ansis Anderson
Muktha Ramesh
Brianna Lebrun

Robotics: Science and Systems

April 24, 2026

1 Introduction and Motivation - Muktha

For a robot to autonomously navigate through its environment, it is essential that it can optimally create path plans, estimate its position and orientation, and use this information to follow the plan. By utilizing known maps of the environment and real-time LiDAR sensor data, our robot efficiently and robustly solves the problem of pose estimation using Monte Carlo Localization (MCL). Our robot performs path planning using the A* algorithm and follows this plan with Pure Pursuit. This report explains the theory behind these three methods (MCL, A*, Pure Pursuit) and how they are implemented in our robotic system. Furthermore, we explore the use of RTAB-Map, a visual SLAM algorithm, to create the maps that MCL relies on, as well as RRT*, which is an alternative to A* for path planning.

2 Monte Carlo Localization Approach

MCL is a localization method known for balancing accuracy, noise robustness, and real-time efficiency. It works as a particle filter, representing the robot's pose by an adjustable number of particles. An initial set of particles gets updated at each time step by a motion model that accounts for the robot's movement and noise in the system. The likelihood of each of these particles is then determined

by the sensor model, based on real-time LiDAR data. The set of particles gets resampled based on this likelihood, which causes the distribution to converge over time. The estimate of the robot's pose is then given by the average pose estimate of the particles.

2.1 Motion Model

Given the previous pose of the particles $s_{t-1} = (x_{t-1}, y_{t-1}, \theta_{t-1})$, and the robot's odometry data $ds = (dx, dy, d\theta)$, the motion model gives the updated pose of the particles $s_t = (x_t, y_t, \theta_t)$.

To account for noise from the odometry data and other environmental factors we introduce:

$$\begin{aligned} dx_{noise} &\sim \mathcal{N}(0, \sigma_x^2) \\ dy_{noise} &\sim \mathcal{N}(0, \sigma_y^2) \\ d\theta_{noise} &\sim \mathcal{N}(0, \sigma_\theta^2) \end{aligned} \tag{1}$$

Gaussian noise is chosen as motion noise arises from many small independent factors. The same amount of noise might not affect x, y, θ, so it is desirable to choose the spread for each of these independently.

Then our updated poses are:

$$\theta_t = \theta_{t-1} + d\theta + d\theta_{noise} \tag{2}$$

$$\begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} x_{t-1} \\ y_{t-1} \end{bmatrix} + R_{\theta_t} \begin{bmatrix} dx + dx_{noise} \\ dy + dy_{noise} \end{bmatrix} \tag{3}$$

Where R_{θ_t} is the standard 2D rotation matrix for angle θ_t.

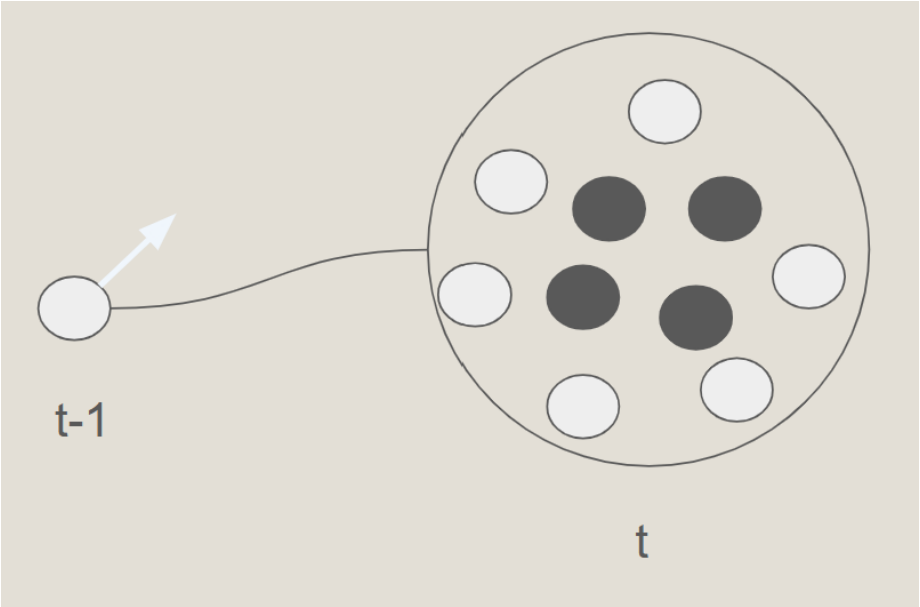

Figure 1: A representation of a MCL particle being updated over time according to the motion model. The more probable particles after the updating are shaded darker than the less probable particles. By adding Gaussian noise to the system, the pose of the particle after the motion model update is not deterministic.

2.2 Sensor Model

Given the current set of particle poses s_k, the n range measurements from the robot LiDAR data $z_k^{(1)}, ..., z_k^{(n)}$, and the environment map m, the sensor model gives the likelihood that the LiDAR data measurement was taken from a particle's pose:

$$p(z_k|s_k, m) = p(z_k^{(1)}, ..., z_k^{(n)}|s_k, m) = \prod_{i=1}^{n} p(z_k^{(i)}|s_k, m) \tag{4}$$

Here n, the ray count per particle, is an empirically chosen parameter.

$p(z_k^{(i)}|s_k, m)$ is the weighted sum of 4 probabilities:

1. Probability of detecting a known obstacle in the map:

$$p_{hit}(z_k^{(i)}|s_k, m) = \begin{cases} \eta \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z_k^{(i)}-d)^2}{2\sigma^2}\right) & \text{if} \quad 0 \leq z_k^{(i)} \leq z_{max} \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

   where η is a normalization constant

2. Probability of a short measurement (detecting an unknown object):

$$p_{short}\left(z_k^{(i)}|s_k,m\right) = \frac{2}{d}\begin{cases}1-\frac{z_k^{(i)}}{d} & \text{if} \quad 0 \leq z_k^{(i)} \leq d \text{ and } d \neq 0\\ 0 & \text{otherwise}\end{cases} \tag{6}$$

3. Probability of a missed measurement:

$$p_{max}(z_k^{(i)}|s_k,m) = \begin{cases}\frac{1}{\epsilon} & \text{if} \quad z_{max}-\epsilon \leq z_k^{(i)} \leq z_{max}\\ 0 & \text{otherwise}\end{cases} \tag{7}$$

4. Probability of a completely random measurement:

$$p_{rand}(z_k^{(i)}|s_k,m) = \begin{cases}\frac{1}{z_{max}} & \text{if} \quad 0 \leq z_k^{(i)} \leq z_{max}\\ 0 & \text{otherwise}\end{cases} \tag{8}$$

We have: $\alpha_{hit}, \alpha_{short}, \alpha_{max}, \alpha_{rand}$:

$$\begin{aligned} p(z_k^{(i)}|s_k,m) = \quad & \alpha_{hit}\cdot p_{hit}(z_k^{(i)}|s_k,m) + \alpha_{short}\cdot p_{short}(z_k^{(i)}|s_k,m) \\ & + \alpha_{max}\cdot p_{max}(z_k^{(i)}|s_k,m) + \alpha_{rand}\cdot p_{rand}(z_k^{(i)}|s_k,m) \end{aligned} \tag{9}$$

The values for our system: $\alpha_{hit} = 0.74$, $\alpha_{short} = 0.07$, $\alpha_{max} = 0.07$, $\alpha_{rand} = 0.12$, $\sigma = 0.4$ m, $z_{max} = 10$ m, and d is the distance calculated via ray-tracing from each particle pose s_k on the map m.

All together we have $p(z_k^{(i)}|s_k,m)$:

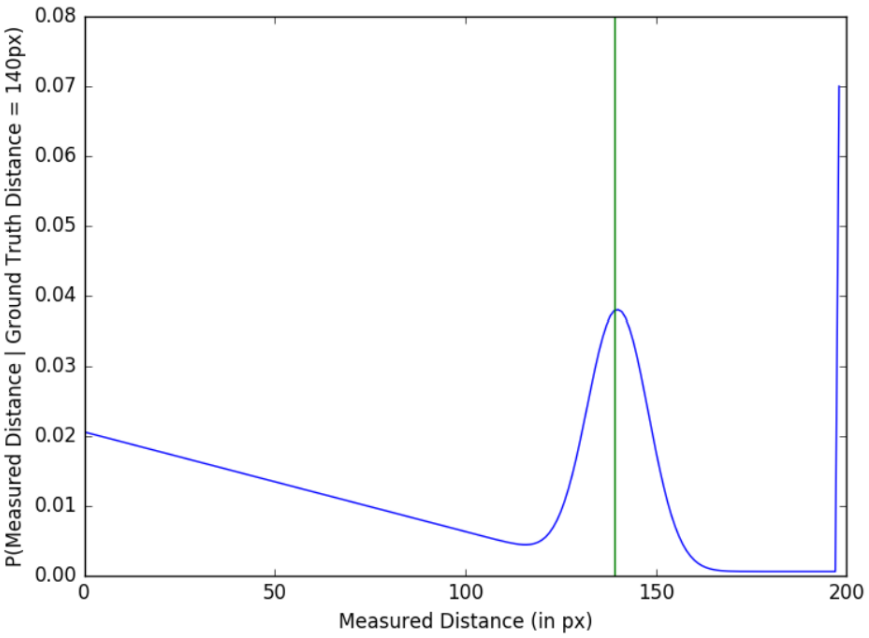

Figure 2: The sensor model probability distribution $p(z_k^{(i)}|s_k, m)$ as a function of distance in pixels. This graph gives a sense of what all the components that make up the probability distribution function look like when combined together.

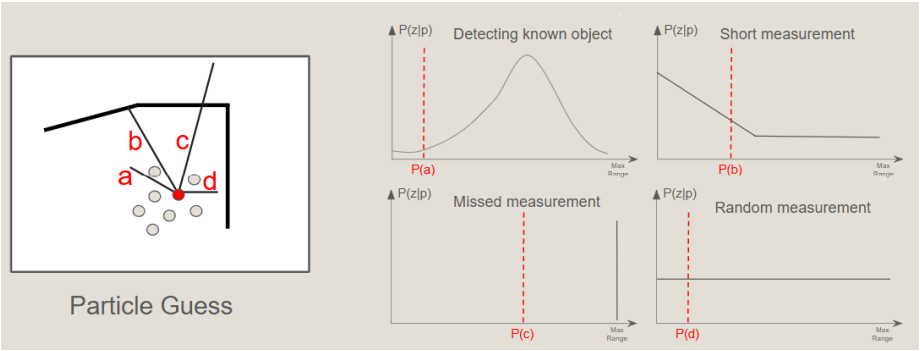

Figure 3: An illustrative example of how the sensor model works. For each particle range, measurements are ray-traced, and their probability is calculated.

2.3 Particle Filter

To finalize the implementation, the motion model is applied on every odometry update, while the sensor model is applied on LiDAR updates. After the weights

of the particles are extracted, the particles are resampled based on these weights, and a slight amount of noise is added. This helps separate the particles, instead of keeping many copies of the same particle until the next motion model update. In practice, the sensor model does not run at the same speed as the LiDAR; however, since the motion model can provide a good estimate for short amounts of time, this delay does not have a major localization quality impact.

In order to give our robot an initial guess of the position, we designed two different particle initialization modes - one creates particles around a given point, allowing for fast convergence, and the other creates the particles equally over the whole map to attempt global localization. In the final implementation, we removed the global initialization mode, since on the real robot, the hardware could not handle enough particles to converge reliably.

To extract the estimated position, we average the x and y positions of all the particles. Extracting the average angle is slightly trickier as it is a circular quantity; however, techniques for it are well-documented. We convert all the angles to points on the unit circle, average these points, and take the angle of the resulting point.

One thing to note is that if there is a minor difference in the weights of the particles (even less than 5%), this difference is repeatedly applied with every sensor model update, removing everything but the most likely particle very quickly. To combat this, we implemented a softening factor, and take the respective power root of all of the weights. This effectively multiplies the time it takes for other particles to disappear by the softening factor, smoothening out the distribution.

3 Evaluation in Simulation

To set the adjustable parameters of our implementation of MCL and test its effectiveness we first turned to evaluation in simulation.

3.1 Relevant Metrics

The major inputs into our implementation of MCL include the particle count, the number of LiDAR rays to down-sample to, and the noise in our motion model. These inputs could affect MCL performance, which could be evaluated with metrics like cross-track error, convergence time, and publishing rate.

1. **Cross-Track Error:** Cross-track error is defined as the Euclidean distance between the ground truth in the simulation and the converged localized position.

2. **Convergence Time:** Convergence time is defined as the time MCL takes to converge to the localized position after an initial pose is set.

3. **Publishing Rate:** Publishing rate is defined as the frequency at which our implementation of MCL updates its guess on a new position.

Minimizing cross-track error improves the accuracy of the robot's estimate of its position, minimizing convergence time allows the robot to have a localization more consistently while navigating, and maximizing publishing rate allows for better position estimates as it more closely matches the robot's position in real time.

3.2 Evaluation Method

In every trial, rosbags are recorded where the robot is sent to an initial pose and then continues to drive down a hallway while localizing. A default parameter set of 200 particles, 100 rays, $\sigma_x = 0.06$, $\sigma_y = 0.02$, and $\sigma_\theta = 0.1$ was kept. Each new change in input parameter required three trials.

To evaluate the effect of particle count, particle count was varied while the rest of the default parameters were kept constant. Ray count and noise were varied similarly.

The evaluation script plotted localization errors, in terms of Euclidean distance between ground truth and localized poses. Figure 4 shows an example trial. Cross-track error was determined by taking the mean of the last 20% of the recorded localization errors. Convergence time was determined by fitting an exponential decay function to the localization errors, and setting a default value of 0.1s when convergence is close to instantaneous. Publishing rate was determined by the frequency of the pose messages in the played back rosbags.

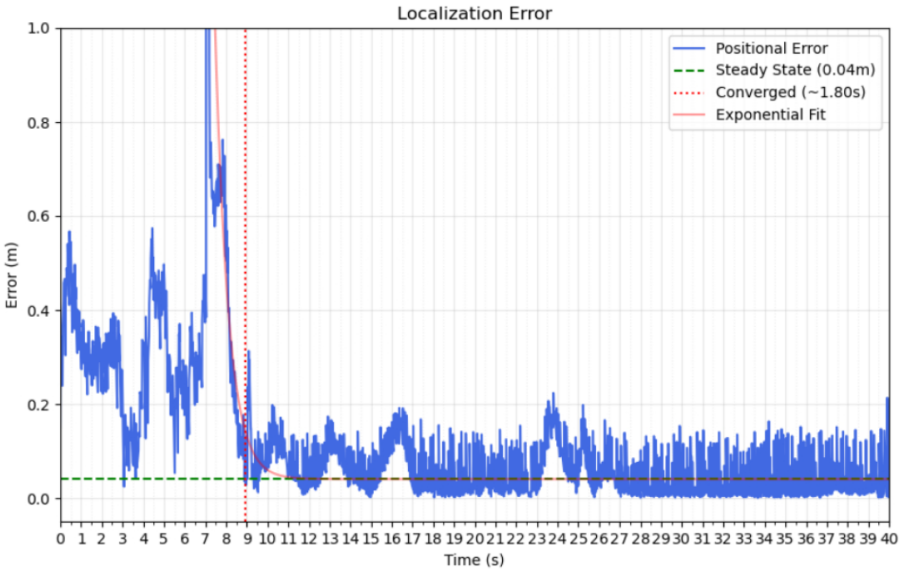

Figure 4: Localized position error. The graph is plotted as the robot drives in simulation for one trial. Cross-track error is determined from the converged last 20% of the simulation time. Convergence time is determined after the peak in error, which corresponds to when the initial pose is set. This particular graph demonstrates how our implementation of MCL is able to quickly converge with satisfactory accuracy.

Limitations in this evaluation method included a limited number of cases tested for each relationship, a limited range of values tested, since it was difficult to know which values were interesting before testing began, and inconsistencies like different computer performance and slightly different initial poses while the rosbags were collected.

3.3 Results

Figures 5 through 9 depict the results of the simulation evaluation. Due to limitations in the evaluation method (large uncertainty, not enough data), it made it difficult to definitively claim effects regarding motion model noise on our metrics.

From what we can see, increasing particle count decreases the time needed to converge and decreases cross-track error, but also decreases publishing rate. Increasing ray count also decreases publishing rate, since more computation is needed for each iteration. Although it is hard to see an overall relationship for ray count and cross-track error, the very high uncertainty at the low ray count of 10 is notable. At a ray count of 10, the high publishing rate can sometimes allow for lower cross-track error, as the MCL updates more often, but local-

ization isn't as reliable. Also, there are particularly large uncertainties for the relationships between noise and publishing rate or cross-track error.

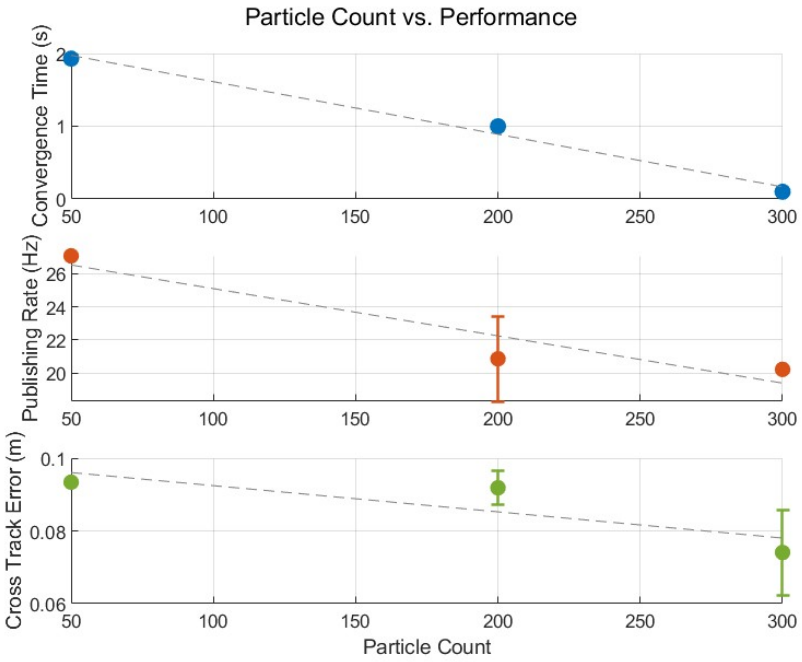

Figure 5: Graphs showing how varying particle count in simulation affects convergence time, publishing rate, and cross-track error. We find that increasing particle count decreases convergence time, publishing rate, and cross-track error. An increased particle count is associated with an increase in performance.

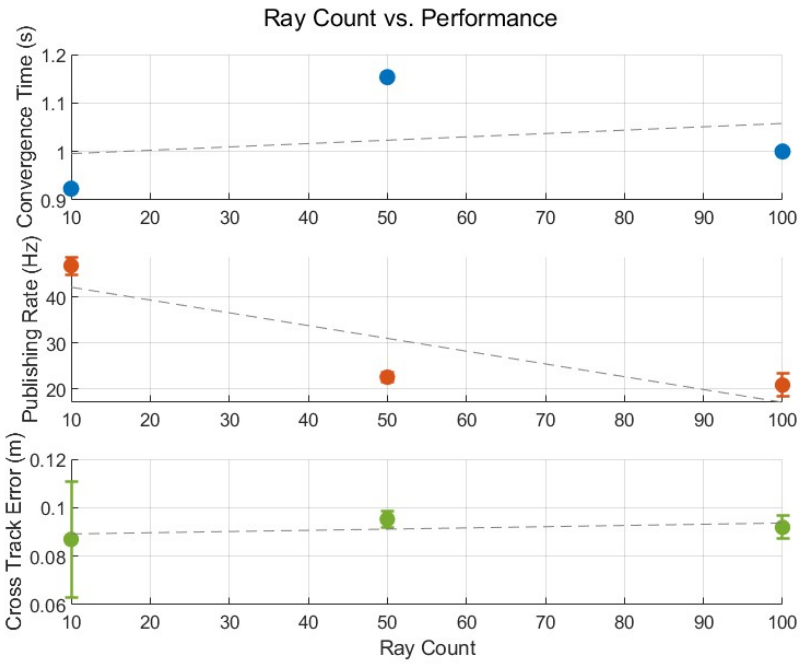

Figure 6: Graphs showing how varying ray counts affects convergence time, publishing rate, and cross-track error. Increasing ray count decreases publishing rate. The effects on convergence time and cross-track error is more uncertain. There is particularly high uncertainty for cross-track error at very low ray counts, partly due to very high publishing rates lowering cross-track error.

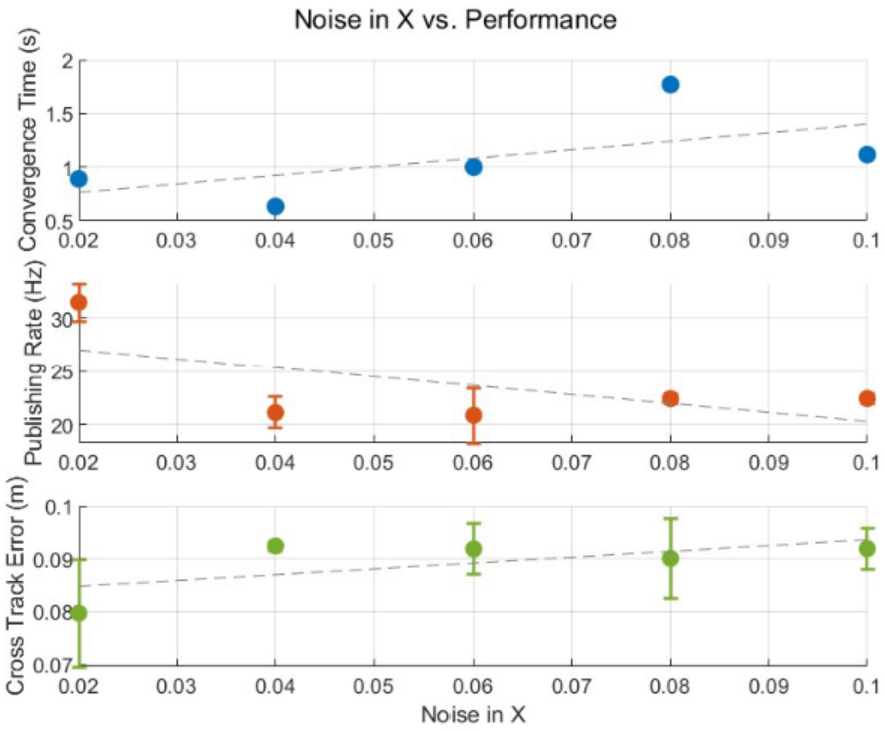

Figure 7: Graphs showing how varying noise in x affects convergence time, publishing rate, and cross-track error. There are high uncertainties for the effect of noise on evaluated metrics. The limited range in tested input noise values also prevent clear relationships from being seen.

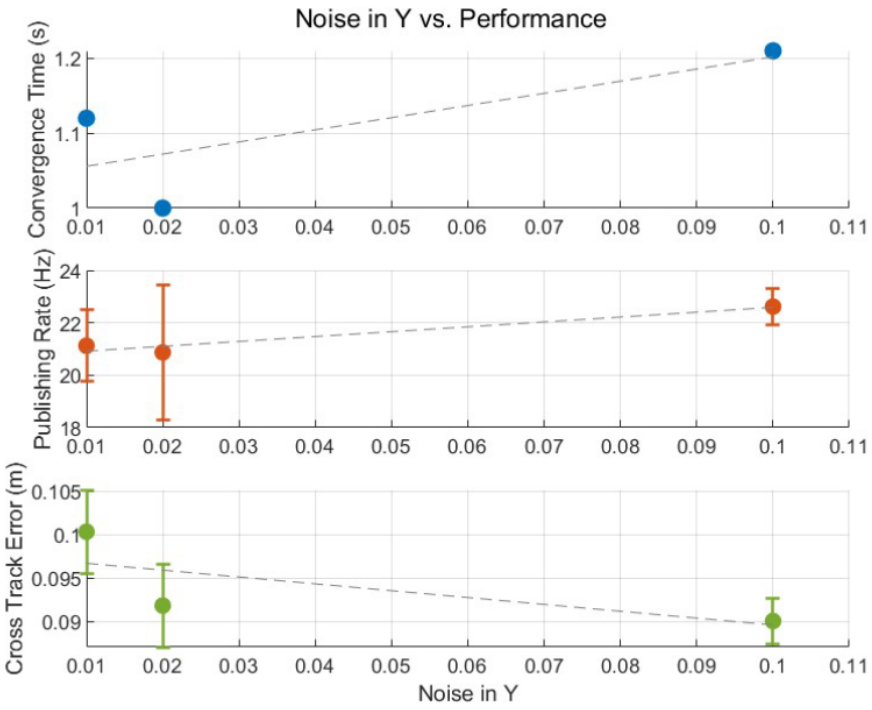

Figure 8: Graphs showing how varying noise in y affects convergence time, publishing rate, and cross-track error. There are high uncertainties for the effect of noise on evaluated metrics. The limited range in tested input noise values also prevent clear relationships from being seen.

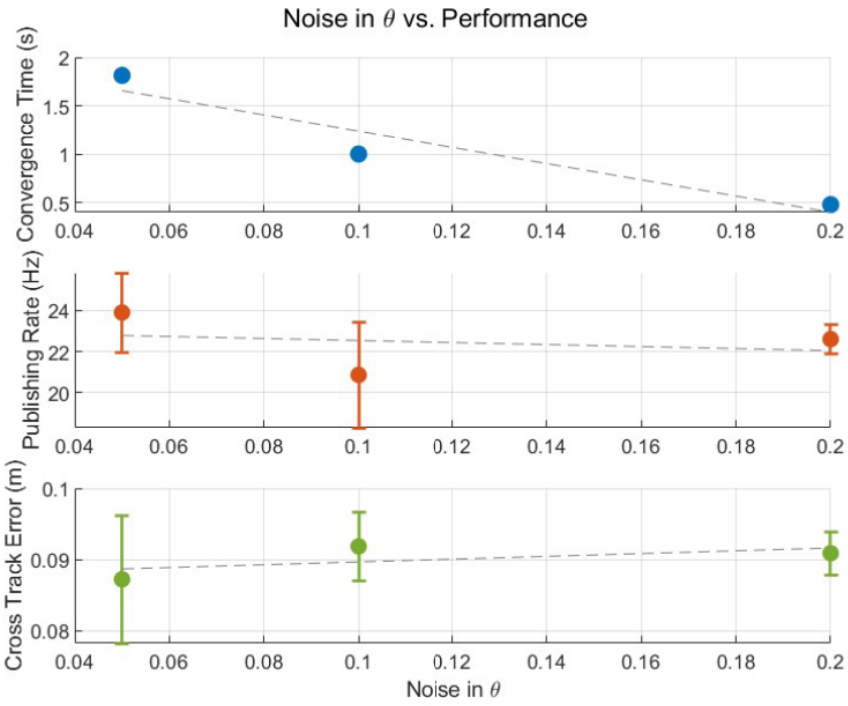

Figure 9: Graphs showing how varying noise in θ affects convergence time, publishing rate, and cross-track error. There are high uncertainties for the effect of noise on evaluated metrics. The limited range in tested input noise values also prevent clear relationships from being seen.

4 Evaluation on Hardware

4.1 Evaluation IRL

Next, we evaluated how well our particle filter performs in a real-world setting. This involves accounting for real-world challenges such as wheel slippage, sensor inaccuracies, and environmental variability. These factors can lead to more divergence between the predicted and actual robot states, making robust localization more difficult than in simulation.

First, we conducted an initial round of empirical testing to identify a set of parameters that allowed the robot to localize reasonably. This let us establish a baseline configuration for the motion model.

After determining a working baseline, we performed a more systematic evaluation by varying key parameters. Additionally, we tested different particle counts

and ray counts to see how they affected the following metrics.

4.2 Metrics

Unlike simulation, real-world evaluation does not provide access to ground-truth pose, preventing direct computation of localization error (e.g., cross-track error). The following metrics were used in evaluating the IRL localization:

1. **End-Pose Accuracy:** Whether the final estimated pose of the robot is consistent with its true location at the end of a trajectory. Specifically, whether the MCL estimation of the final pose is within 2.5 meters of the true final pose.
2. **Particle Variance After Convergence:** The spread of the particle set after the filter has converged. (The model has converged when the variance has reached less than 0.35 in our evaluation.)
3. **Publishing Rate:** The frequency at which pose estimates are produced by the particle filter. This rate reflects the computational efficiency of the system and its suitability for real-time deployment, as well as indirectly impacts localization accuracy while moving, as shown from the simulation.

In addition to these quantitative metrics, we qualitatively evaluated each trial by visualizing the particle distribution and estimated trajectory. This ensured that the filter exhibited expected behavior, such as convergence to the correct region and stability over time.

4.3 Evaluation Method

In order to ensure that the only variables at play were the parameters we were testing for, we ran MCL on rosbags collected from the robot. We utilized 4 unique test runs, and tested each parameter on each corridor 3 times (12 tests each).

For motion model parameter settings, our default settings were 0.1 for the x and y standard deviations, and 0.08 for our theta standard deviation. While keeping the rest of the parameters the same, we individually varied our x and y to be [0.01, 0.03, 0.1, 1], and theta to be in [0.02, 0.04, 0.08, 0.1]. After we got our best setting for x, y, and theta, we also tried the combination of the best standard deviation values for all three.

Finally, we also had attempted a variety of particle/ray counts. A few of these (# particle, # ray) settings included (50, 150), (100, 150), (100, 99), (200, 99), and (2000, 10).

Each configuration is evaluated based on success rate, post-convergence variance, and publishing rate. Results highlight trade-offs between localization accuracy, robustness, and computational efficiency across environments. The two highlighted configurations correspond to the best-performing settings in terms of end-pose accuracy.

4.4 Analysis

Table 1: Top 10 MCL parameter configurations ranked across four corridor environments.

Top 10 settings overall								
rank	**particles**	**rays**	**sigma_x**	**sigma_y**	**sigma_theta**	**success_rate**	**var_after_convergance**	**publish_rate**
1	2000	10	1	0.1	0.08	0.917	didn't converge	66.924
2	200	150	0.03	0.01	0.04	0.75	0.109	89.871
3	50	150	0.03	0.01	0.04	0.667	0.112	90.097
4	100	150	0.03	0.01	0.04	0.583	0.111	90.041
5	2000	10	0.1	1	0.08	0.583	didn't converge	70.194
6	100	99	0.03	0.01	0.04	0.5	0.121	90.095
7	200	99	0.03	0.01	0.04	0.5	0.125	90.035
8	2000	10	0.1	0.1	0.08	0.5	0.318	68.422
9	2000	10	0.1	0.1	0.04	0.417	0.313	69.165
10	2000	10	0.1	0.1	0.08	0.417	0.323	68.459

Based on end pose accuracy alone, which seems to be the most important measure, it seems as though the yellow setting is the best. However, upon further inspection, the yellow setting struggled to do well in featureless areas such as long hallways. Due to the many particles and high uncertainties, this setting would lead to the robot being very uncertain about its position, which we can see as the localization didn't converge (variance never went below 0.35).

On the other hand, the blue setting did much better with localization in featureless sections. Once converged, the blue setting would output meaningful data throughout the long hallways. Although it did worse in end pose accuracy, it was much more useful in the middle of the robot trajectory, and did better in the other two metrics.

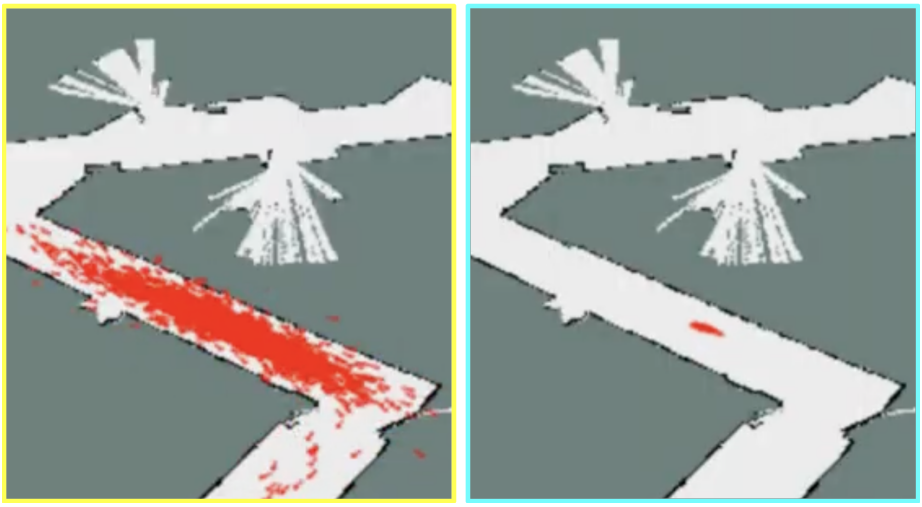

Figure 10: Comparison of particle filter behavior under different parameter settings. The left image corresponds to the yellow setting, the setting with the best end pose accuracy. It shows a highly dispersed particle distribution indicating poor convergence and high localization uncertainty in featureless areas such as a long hallway. The right image corresponds to the blue setting which had the second best end pose accuracy. It shows successful convergence, where particles collapse tightly around a consistent pose estimate even in a long hallway.

Top 10 settings for corridor1								
rank	**particles**	**rays**	**sigma_x**	**sigma_y**	**sigma_theta**	**success_rate**	**var_after_convergance**	**publish_rate**
1	50	150	0.03	0.01	0.04	1	0.101	90.093
2	200	150	0.03	0.01	0.04	1	0.104	89.898
3	100	150	0.03	0.01	0.04	1	0.104	90.052
4	2000	10	0.03	0.01	0.04	1	0.231	68.187
5	2000	10	0.1	0.05	0.08	1	0.337	68.112
6	2000	10	0.1	1	0.08	1	didn't converge	69.797
7	2000	10	0.1	0.1	0.08	1	didn't converge	68.275
8	2000	10	0.1	0.1	0.04	1	didn't converge	67.665
9	2000	10	1	0.1	0.08	1	didn't converge	67.376
10	2000	10	0.1	0.1	0.02	1	didn't converge	67.366
Top 10 settings for corridor2								
rank	**particles**	**rays**	**sigma_x**	**sigma_y**	**sigma_theta**	**success_rate**	**var_after_convergance**	**publish_rate**
1	100	99	0.03	0.01	0.04	1	0.104	90.204
2	200	99	0.03	0.01	0.04	1	0.108	90.159
3	100	150	0.03	0.01	0.04	0.667	0.099	90.149
4	200	150	0.03	0.01	0.04	0.667	0.103	89.941
5	50	150	0.03	0.01	0.04	0.667	0.132	90.125
6	2000	10	1	0.1	0.08	0.667	didn't converge	64.564
7	2000	10	0.1	0.1	0.08	0.333	0.334	66.448
8	2000	10	0.1	0.05	0.08	0.333	0.384	64.64
9	2000	10	0.1	1	0.08	0.333	n/a	67.778
10	50	99	0.03	0.01	0.04	0	0.099	90.223
Top 10 settings for corridor3								
rank	**particles**	**rays**	**sigma_x**	**sigma_y**	**sigma_theta**	**success_rate**	**var_after_convergance**	**publish_rate**
1	2000	10	0.1	0.01	0.08	1	0.433	67.778
2	2000	10	0.1	1	0.08	1	didn't converge	71.521
3	2000	10	1	0.1	0.08	1	didn't converge	66.55
4	2000	10	0.1	0.1	0.04	0.667	0.309	69.872
5	2000	10	0.1	0.1	0.08	0.667	0.31	67.022
6	2000	10	0.1	0.1	0.08	0.667	0.31	68.997
7	50	150	0.03	0.01	0.04	0.333	0.102	90.061
8	200	150	0.03	0.01	0.04	0.333	0.111	89.687
9	100	150	0.03	0.01	0.04	0.333	0.117	89.888
10	200	99	0.03	0.01	0.04	0.333	0.13	89.852
Top 10 settings for corridor4								
rank	**particles**	**rays**	**sigma_x**	**sigma_y**	**sigma_theta**	**success_rate**	**var_after_convergance**	**publish_rate**
1	200	150	0.03	0.01	0.04	1	0.12	89.959
2	2000	10	1	0.1	0.08	1	didn't converge	69.207
3	50	150	0.03	0.01	0.04	0.667	0.114	90.11
4	100	150	0.03	0.01	0.04	0.333	0.126	90.077
5	100	99	0.03	0.01	0.04	0.333	0.14	90.111
6	50	99	0.03	0.01	0.04	0.333	0.144	90.136
7	200	99	0.03	0.01	0.04	0	0.149	90.085
8	2000	10	0.03	0.01	0.04	0	0.291	69.745
9	2000	10	0.01	0.1	0.08	0	0.317	68.847
10	2000	10	0.1	0.01	0.08	0	0.326	68.157

Figure 11: Comparison of localization performance across different parameter settings in each of the 4 test cases. The results show that parameter choices significantly affect convergence stability, consistency, and robustness differently across environments.

In addition, when ranking by end pose accuracy and then variance, the blue setting actually did better in 3 out of the 4 test runs on average. Thus, we are confident in choosing the blue setting to put onto the robot.

5 Extension to simultaneous localization and mapping (SLAM)

To test out localization in a different environment and verify performance, we did a separate study, capturing our own map and running localization on it. We succeeded in this and achieved successful localization, proving that our implementation is robust to environmental changes.
To do this, we used a graph-based SLAM algorithm provided by the ros rtabmap package. Compared to the MCL algorithm from earlier, the biggest difference is that SLAM continuously constructs and updates an estimate of the map and the path taken so far, attempting to explain past LIDAR, camera, and odometry observations. This is possible due to the algorithm incorporating images and a depth map from the ZED camera to identify repeating landmarks, and using that to adjust for the odometry dead reckoning errors.
The location we chose was the lounge and part of the hallway in East Campus. This proved to be a challenging environment, mostly due to the tight hallway requiring several turns in each loop, heavily offsetting the dead-reckoning odometry. Driving extremely slowly, as well as adjusting the path to incorporate an easily repeatable loop, improved convergence speeds.

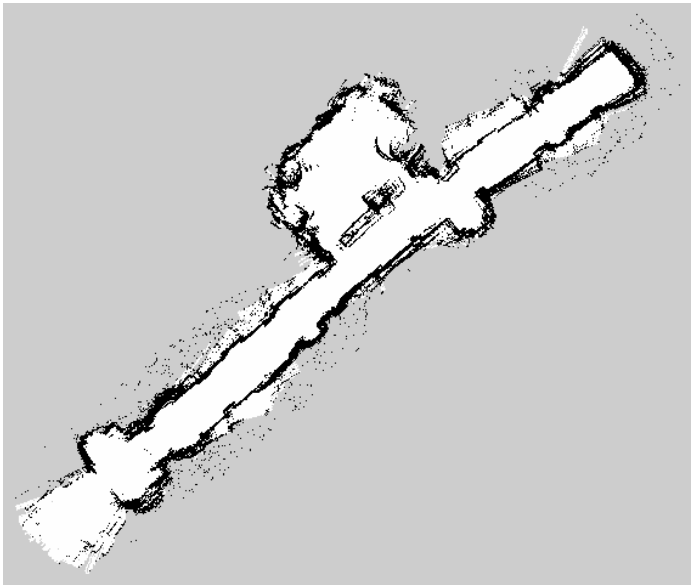

Figure 12: The final map captured after a 15-minute SLAM session. Slight curvature is still present in the hallway, which could be addressed by more loops; it does not have an impact on performance in this case.

To verify the captured map, we ran localization and visually compared the

estimated position to the recorded video of the robot driving. As expected, the particle filter tracked the position very accurately, since the map had just been captured, and there weren't any unexpected obstacles or differences between the map and the real world. Due to this, we did not consider this map for evaluations, focusing on a robust evaluation on the more difficult Stata Center basement map.

6 Path Planning - Ansis + Muktha

6.1 Grid Based Approach - Ansis

For the grid-based approach, we chose A* for its speed and efficiency compared to other alternatives. A* explores grid-based paths by checking the 8 direct neighbors of each node, and uses a heuristic, in this case, Euclidean distance to goal, to prioritize paths moving towards the goal.
While A* paths are guaranteed to be optimal under the chosen constraints, the restriction to 45-degree angles results in many unnecessary turns and an overall path length increase. To combat this, we apply simple line-of-sight smoothing on the completed path, removing redundant points where possible.

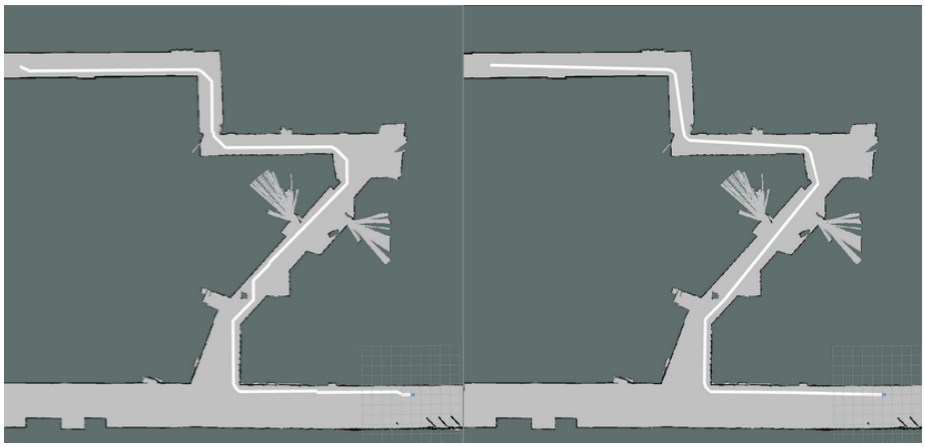

Figure 13: Comparison of the original and smoothed path. While not guaranteed to be optimal under any-angle assumptions, the smoothed path is shorter and takes fewer redundant turns, which makes pure pursuit on it easier as well.

6.2 Sampling Approach - Muktha

We used Rapidly-exploring Random Tree* (or RRT*) as our sampling approach. RRT* continuously samples points across the map to find a path from the start to the end point, while also improving path optimality over time. As the algorithm progresses, it constructs a tree of feasible routes throughout the map.

The algorithm proceeds as follows:

1. Sample a random point from the map.
2. Find the nearest node in the existing tree and attempt to extend toward the sampled point. If the connection collides with an obstacle, discard it.
3. Identify other nearby nodes within a neighborhood and choose the parent that minimizes the path length from the start.
4. Add the new node to the tree.
5. (Rewiring step) For each nearby node, check whether going through the new node reduces its cost. If so, update its parent to improve the overall tree.
6. Repeat.

To improve convergence, we bias sampling toward the goal and near obstacles.

Once a feasible path is found, continuing to run the algorithm and biasing sampling near the current best path further improves optimality.

7 Evaluation in Simulation - Tissany

7.1 Metrics

Our chosen metrics for the path planner included the following:

- Time
- Length of Path
- Number of Waypoints
- Smoothness

The metric of time was chosen because RRT* is known to be probabilistically optimal, so it could run forever. Comparatively, A* is expected to finish a lot faster. In the later evaluation method, we will try to keep the times similar so that the other metrics can be compared. Shorter lengths of paths and fewer waypoints are more ideal. A smoother path is also more ideal as it could be easier for the robot to follow in real life.

7.2 Evaluation Method

In order to evaluate the two path planners, six points were chosen around the map of Stata basement. The same points are clicked for both algorithms, and the evaluation script calculates each metric for each point, then outputs the average of each metric. Since RRT* is probabilistic and does not end on its own after a

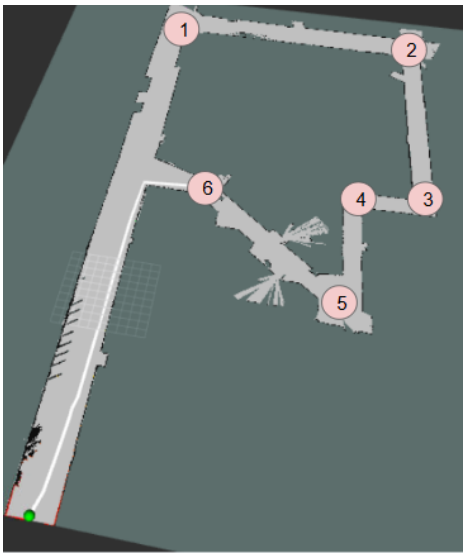

Figure 14: Map of Stata Basement with the six chosen points to evaluate A* and RRT*

set time, the time it took to path plan during the evaluation process was set to roughly the same time that A* took, so that other metrics can be understood based on that forced limit. This wasn't done perfectly in our evaluation, so the average time that RRT* took was still 22 seconds more than A*.

7.3 Results

Table 2: Comparison of A* and RRT* Path Planning Performance

Metric	**A***	**RRT***
Time (s)	4.34	4.56
Length (m)	82.29	91.30
Waypoints (WP)	8.8	11.8
Smoothness	0.324	0.268

Table 2 shows the results of evaluating the two path planning algorithms. For similar times taken, the average length and number of waypoints of RRT* was greater than those of A*. Our RRT* planner was smoother than our A* planner, even with smoothing, likely because of the rewiring step in RRT* that isn't restricted to a grid structure.

8 Path Following with Pure Pursuit - Jeryl

8.1 Modifications to Standard Pure Pursuit

The planned path is represented by a sequence of 2D points:

$$\mathbf{p}_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \in \mathbb{R}^2 \tag{10}$$

These points are broken into segments:

$$\mathbf{s}_i = \mathbf{p}_{i+1} - \mathbf{p}_i, \qquad \hat{s}_i = \frac{\mathbf{s}_i}{\|\mathbf{s}_i\|} \tag{11}$$

The robot is concerned with following the path, starting at the segment nearest to the robot position $\mathbf{q}_t$ (estimated via MCL):

$$t_i = \text{clip}\left(\frac{-\mathbf{s}_i \cdot (\mathbf{p}_i - \mathbf{q})}{\|\mathbf{s}_i\|^2},\ 0,\ 1 \right) \tag{12}$$

$$\mathbf{r}_i = \mathbf{p}_i + t_i\, \mathbf{s}_i \tag{13}$$

$$i^* = \arg\min_i\ \|\mathbf{r}_i - \mathbf{q}\|_2 \tag{14}$$

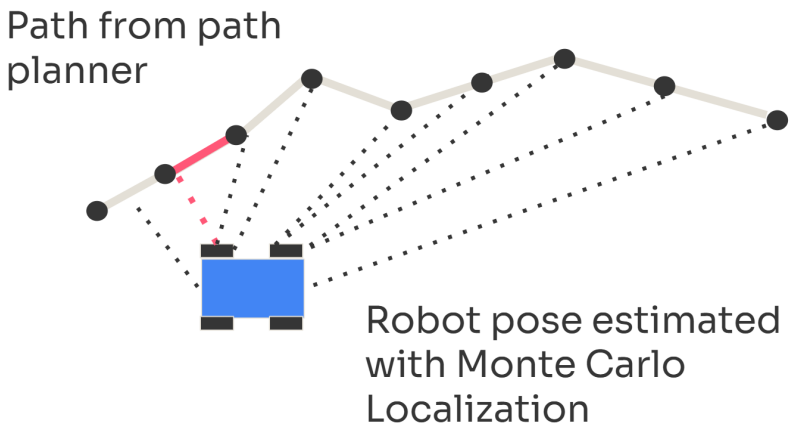

Figure 15: The segment labeled $\mathbf{s}_{i^*}$ (shown in red here) is the closest to the estimated position of the robot $\mathbf{q}_t$. The N segments after $\mathbf{s}_{i^*}$ are used to estimate the curvature of the path, allowing for a dynamically updated lookahead distance.

The upcoming curvature of the path κ, is estimated as a decay weighted average of the angles θ_i, between the next N segments, with decay rate λ:

$$\theta_i = \arccos\left(\hat{s}_i \cdot \hat{s}_{i+1}\right) \tag{15}$$

$$w_i = \lambda^i \tag{16}$$

$$\kappa = \frac{\sum_{i=i^*}^{i^*+N-1} w_i\, \theta_i}{\sum_{i=i^*}^{i^*+N-1} w_i} \tag{17}$$

The dynamic lookahead distance L_1, scales the base lookahead distance L_0 inversely with the upcoming curvature κ and gain k_κ.

$$L_1 = \frac{L_0}{1 + k_\kappa\, \kappa} \tag{18}$$

The lookahead point is then the intersection of a circle of radius L_1 centered at the robot position $\mathbf{q}_t$ and the path, and standard pure pursuit is applied.

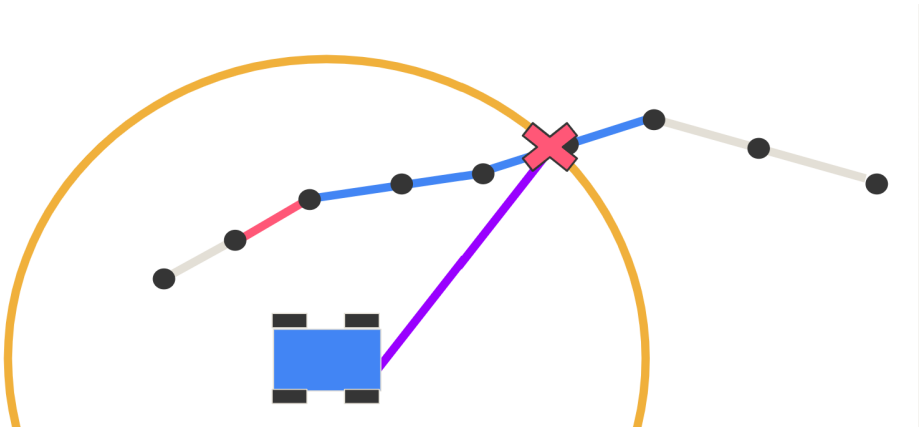

Figure 16: For upcoming segments with less curvature, a smaller κ is computed, leading to a larger lookahead distance L_1. This allows the robot to have tighter path following for turns and corners.

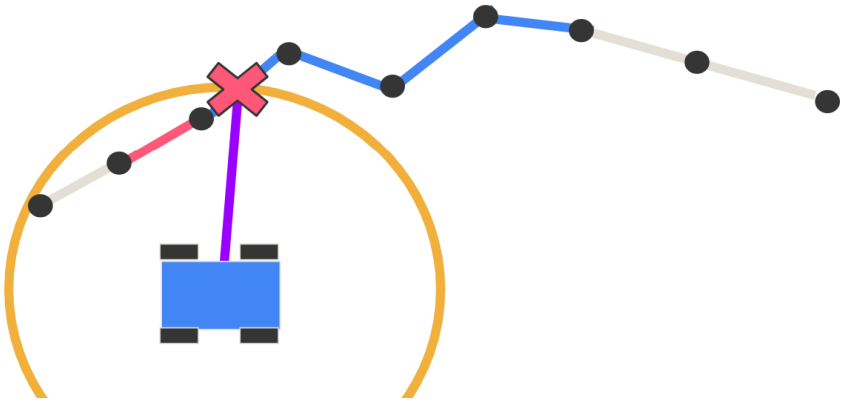

Figure 17: For upcoming segments with more curvature, a larger κ is computed, leading to a smaller lookahead distance L_1. This allows the robot to have smoother and more stable path following with less oscillations for straightaways.

8.2 Evaluation Method

To evaluate the performance of the path follower in simulation, a planned path consisting of two goal pose was set and the cross-track error, heading error, and steering angle were analyzed.

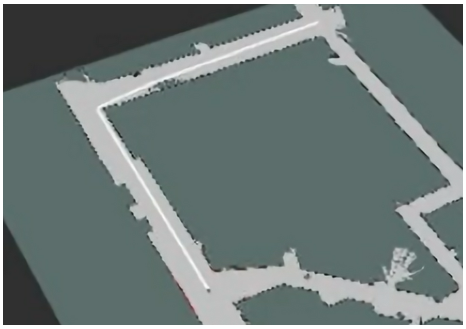

Figure 18: The first goal pose of the planned path to evaluate path following performance in simulation. This path contains long straightaways and a sharp turn.

Figure 19: The second goal pose of the planned path to evaluate path following performance in simulation. This path contains multiple sharp turns and more curvature in a tighter corridor.

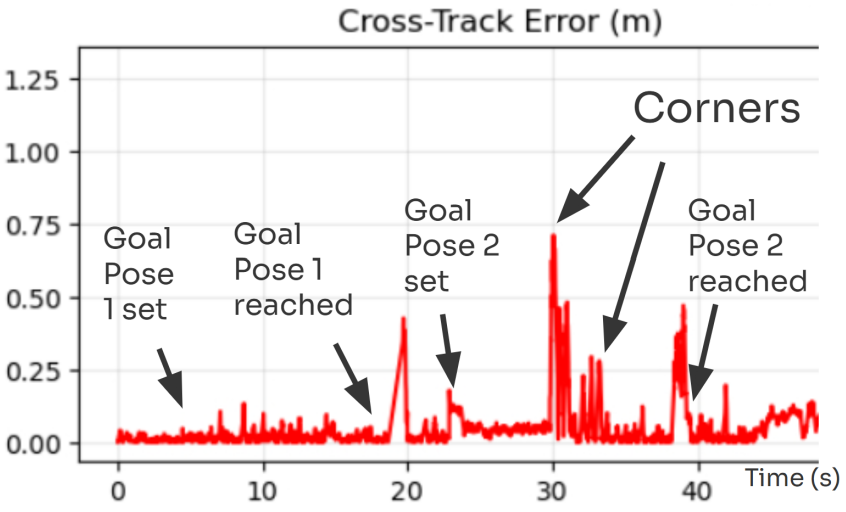

Figure 20: The cross-track error between the robot's position and the planned path over the two goal poses tested in simulation. Here the robot was able to stay within 1 meter of the path, even while turning tight corners.

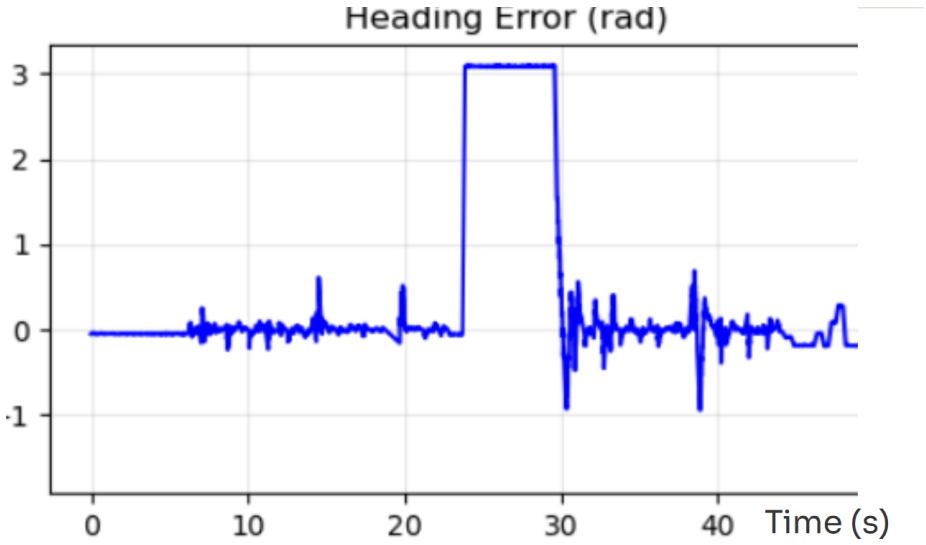

Figure 21: The heading error between the robot and the planned path over the two goal poses tested in simulation. Here the robot stayed very well aligned with the path, even while taking corners. The large heading error from 20 to 30 seconds is due to the setting of the second goal pose.

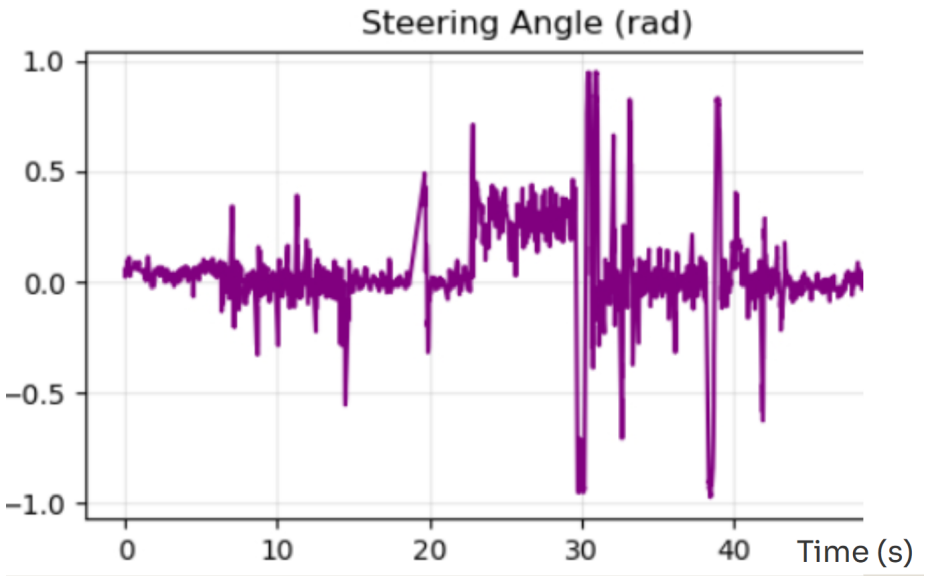

Figure 22: The steering angle of the robot over the two goal poses tested in simulation. Here there are high frequency oscillations which suggests that more tuning of parameters is necessary.

The robot was able to successfully follow paths in simulation, staying within 1 meter even while turning corners. Better parameter selection can fix high frequency oscillations in steering angle and higher cross-track error while turning corners.

9 Evaluation on Robot - Bre

9.1 Metrics

To evaluate our path planning algorithm on the robot, we used the following metrics:

- Length of Path
- Curvature of Path
- Cross-Track Error

We want to ensure our robot is robust across all types of paths, which is why we highlight testing across different path lengths and degrees of curvature. Additionally, we want to evaluate how well the robot performs at following the planned path exactly, which is why we chose the cross-track error.

9.2 Evaluation Method

In order to evaluate our path planner on the robot, we ran a series of tests, with each consisting of three trials. Specifically, these tests were following a short straight path, a long straight path, and going around a turn. For each of these, we qualitatively assess if this robot has reached its end goal and also calculate the cross-track error along each path.

9.3 Results

For both the short straight path and long straight path, we saw success in reaching the target position with minimal cross-track error, as shown in Figures 23 & 24.

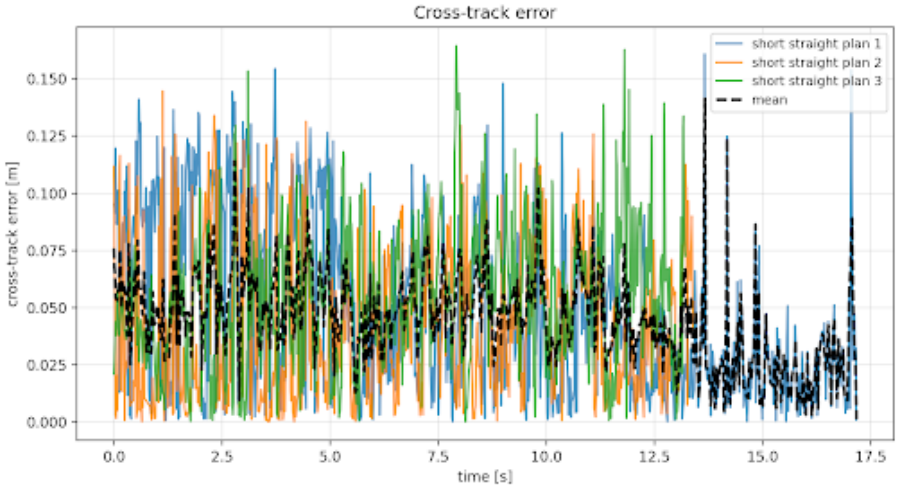

Figure 23: The average and individual cross-track error of 3 trials following a short, straight path

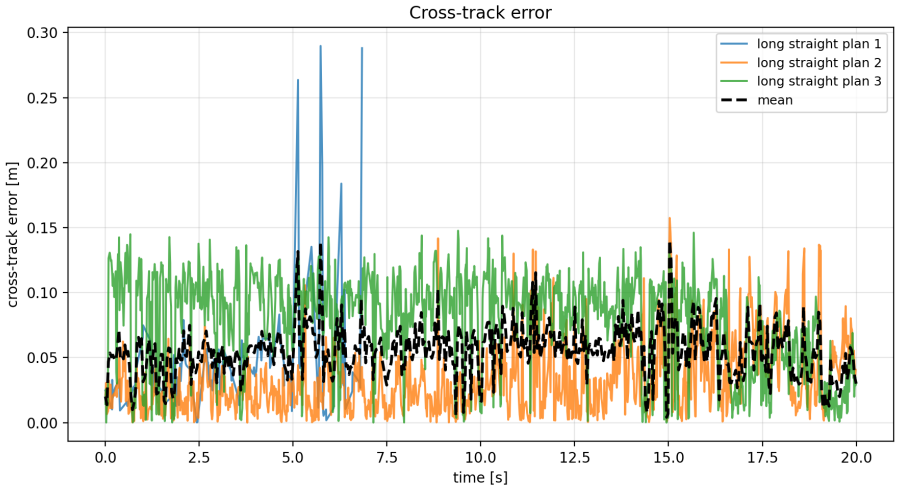

Figure 24: The average and individual cross-track error of 3 trials following a short, straight path

In both cases, we saw that the robot performed well with an average cross-track error of about 0.05 across both experiments. We do, however, see that in the tests for a long straight path, one of the trials failed, showing that our algorithm isn't completely robust.
When it came to testing the robot's ability to handle curves, we found that our

localization algorithm was lacking and may give too much weight to our sensor model. On tight turns, it failed to straighten back out and would instead trigger the safety controller once in close enough range with the wall. Further testing is necessary on the localization algorithm to make the car more robust against turns.

10 Conclusion - Ansis

We built a system that integrates localization, planning, and control, and demonstrated that it works well in real-time. Our implementation of MCL is able to operate with satisfactory convergence rates and accuracy in simulation and on hardware. While featureless sections of the map are still challenging, the selection of parameters allows for reliable localization in most cases. We have additionally shown that maps generated by visual-SLAM algorithms allow our robot to localize in new environments.
For path planning, we explored both A* and RTT, and observed clear tradeoffs. While RTT could produce smoother paths, it had issues with runtime and variability. A* was better suited for real-time use due to its reliability and speed. The pure pursuit controller worked well with the A* paths; however, our implementation is sensitive to minor localization mistakes. For future improvements, we will continue tuning our MCL implementation.

11 Lessons Learned for MCL

11.1 Tissany's lessons learned

I learned that it is important to fully consider how much of the evaluation process to automate before starting. A lot of the uncertainty in the data could likely be mitigated if just one rosbag was collected so that computer performance and initial pose are more constant, but an entirely automated evaluation system could hide scenarios which benefited from seeing each localization error plot individually, like immediate convergence times that didn't work with the exponential decay model. I also learned that error bars can reveal a lot of detail about which relationships can be trusted, especially in cases with sparse data points.

11.2 Jeryl's lessons learned

Through this lab I learned that it can be really helpful to think about evaluation even before working on the implementation. By deciding the kinds of metrics you need beforehand and the types of testing you will be doing it becomes a lot faster to get and analyze the data. I also learned how important it is to redelegate tasks and keep a clear track of the progress of tasks.

11.3 Ansis' lessons learned

As I was focusing on the implementation side, I learned that it is easy to waste time overengineering features that don't actually make it into the final product. I wrote several features to help the robot recover from bad localization and return to a converged state; however, I ended up removing them since the filter updating more frequently and the robot never losing the localization in the first place was way more beneficial. Additionally, seemingly minor things can have a significant performance impact, for example, just publishing particles after every motion model update.

11.4 Muktha's lessons learned

On the technical side, I became even more comfortable with ROS, and especially with dealing with rosbags. I also learned a lot about localization. It was really interesting to see how much drift actually accumulated over time without localization, as well as figuring out how localization actually worked. Learning the quirks of MCL was also interesting, such as how dealing with long hallways (featureless areas) is actually pretty difficult for the robot, and how particle counts, ray counts, and different parameters in the sensor and motion models all contribute to whether the MCL will ultimately converge to the correct location. Even publishing rate ended up mattering, so optimizing that ended up helping accuracy, and tools like numpy were very useful here. When evaluating MCL, I also realized how memory intensive it was. Luckily for the robot, it could discard the particles immediately, but in evaluation I had to stop in the middle twice due to storage issues when holding 2000 particles published at 90hz. On the communication side, I think I learned how useful it is to spend the time we have together making a clear agenda for that meeting as well as the tasks for after the meeting was over. We do this most of the time, but I realized how useful it actually is in order to get things done.

12 Lessons Learned for Path Planning

12.1 Tissany's lessons learned

While we had originally planned for integration of the path planning and the path following code to be done by one person, individually, our timeline had to shift and the result was that this integration step was done in lab, with the people that implemented each part present. This allowed us to be more efficient in the integration step, when it came to on-the-spot debugging of each part. This reminded me of the integration we did in a previous lab, for visual servoing, when integration was also smoother because teammates that completed each part were present.
When we moved onto integration on the real car, it was easier to understand the robot's behavior because multiple situations were already tested for in sim-

ulation. For example, when the path follower failed on the real car and the car started moving in circles, often when it seemed to have reached the goal pose already. I realized that this situation would've been difficult to debug if integration was started in real life. In simulation, though, we had already seen that the path follower would fail more often when the goal pose is set to behind the robot. As such, in real life, the situation of the path follower failing was deduced to be due to localization instances where the robot ended up in front of the goal pose.
Another lesson learned was more generally that if it's hard to do something one way, there are simpler ways to move forwards that should be considered earlier on in order to save time. While I was evaluating the path planners, I was having trouble running one of the path planners when I was trying to have both pieces of code on the same branch. It could be that I did not migrate all of the code relevant to one path planner, but I was having trouble spotting the differences. Afterwards, I decided to evaluate the two path planners on different branches and copied over my evaluation code instead. This process went by a lot smoother. I think this situation was an exercise in understanding which problem to tackle for the same solution.

12.2 Jeryl's lessons learned

In this lab I learned how important it is to consider inputs and outputs of different subsystems when trying to parallelize the development of a bigger system in general. I think that we were able to implement path planning and path following at the same time because we clearly communicated our larger design features beforehand. I think it would be more ideal if all group members were able to help with evaluation of implementation. We might want to do a better job of splitting tasks to make this possible in the future.

12.3 Ansis's lessons learned

It's easy to get lost exploring different approaches and reinventing the wheel, but most of the time that isn't necessary. A* is fast and reliable, and I should have tested it out first, instead of attempting any-angle pathfinding before it (Theta*, Lazy-Theta*, AP-Theta* all did not provide any meaningful improvements and took way too long to justify use). On the communications side, I think we were kind of disorganized this time, mostly due to the quick turnaround time for this lab, and I think this is something we should focus on more in the future.

12.4 Muktha's lessons learned

On the technical side, I learned about RRT*. I also, through a lot of experimentation, learned about what could affect the speed of RRT* for finding a path or optimizing. For instance, prioritizing samples near obstacles and near the end pose is useful to create the first plan, and sampling around the best path is good for optimization. Allowing connections for nodes far away from each other also

helped speed things up and reduce node count faster in the paths. I also realized how important it is to have a shared spec defined early on (especially between path planning algorithms) for easier evaluation and so that other people can work on other parts like the pure pursuit in parallel. I think planning our tasks and defining those specs were some of the most useful things we could do during our team meetings.

12.5 Bre's lessons learned

As I was focused mainly on the evaluation and integration for this lab, an issue I ran into a lot was trying to hypothesize where exactly in the pipeline an error may have arose or further tuning was necessary. So I had to go back and more closely review what other had written to decide what parameters what be best to try out and test next, which wasted a lot of the time I had planned to dedicate. It would've been a lot more beneficial if before going into testing I had taken the time to go over the code others had written and in that case I could have done a better job of gearing the tests towards the specific implementations.
Editor: Ansis Anderson